

Product specification — Switch It

Student final project for **Internet Technologies – Become a Full-Stack Engineer – RUNI CS 2026**.

This document describes the **product as implemented in the current repository**. It does not invent features.

Product overview

Switch It is a phone-first web application (PWA-capable) that coordinates a **direct driver-to-driver parking handoff**.

It is not merely “someone reported that a parking spot may become free.”

The core idea is a temporary match between:

- a **publisher** — the driver leaving a public street parking spot
- a **seeker** — a specific driver trying to take that spot

Together they share timing, vehicle identity, and short-lived confirmation.

Switch It coordinates the two drivers. It does **not** sell, reserve, own, or guarantee a public parking space. Another non-app driver can still take the physical space.

Problem

Finding street parking in dense cities is slow and uncertain. Drivers circle blocks without knowing whether a space will open, and when someone is about to leave, there is usually no reliable way to coordinate with a nearby driver who needs that space.

The practical problem Switch It addresses is **coordination and timing**, not legal reservation of public parking:

- Reduce wasted search time by matching a leaving driver with a nearby seeking driver.
 - Make the exchange more predictable with shared departure timing, a short active handoff window, vehicle identification, and confirmation.
 - Create a lightweight incentive (virtual credits) so successful handoffs are rewarded without requiring payments in the MVP.
-

Users

Publisher (leaving driver)

Shares a parking spot they are about to leave:

1. Sets up an account and vehicle profile.
2. Opens **Share a spot**, confirms location, chooses departure between **Now and 10 minutes**.
3. Waits for a seeker (or starts early with **I'm leaving now**).
4. During the active handoff, may extend once by **+2 minutes** (when a seeker is claimed).
5. Verifies the arriving seeker using vehicle details and the **last 2 digits** of the seeker's stored plate.

6. Completes or cancels; credits move only on successful completion.

Seeker (driver looking for parking)

Looks for an available handoff:

1. Sets up an account and vehicle profile.
2. Opens **Find parking**, sees available spots on a map.
3. Claims a spot (must be within ~1500 m aerial distance; needs ≥ 1 credit balance — claim does **not** debit).
4. Navigates via Waze / Google Maps / Apple Maps; during an active claim the app starts **mandatory** live-location sharing (subject to device permission / GPS availability).
5. Can release/cancel with a structured reason (no credit movement).
6. Waits for publisher confirmation; successful completion debits 1 credit.

The same authenticated person can act as publisher or seeker in different sessions, subject to product rules (one open published spot at a time; one active claim at a time).

Customer

Current MVP: B2C driver-facing product. Monetization and a paying-customer model are **outside the current project scope**.

Role	Status in MVP
End users (drivers)	Implemented
Paying customer (city, operator, ads, subscriptions)	Not currently implemented
Payments	Not currently implemented
Credits	Virtual points only — not money

Honest framing for the course: the user of the academic product is the **driver**. Possible future commercial models are hypotheses only and are not claimed as implemented.

Business value

Reasonable value claims supported by the product design:

- **Reduced search time** for seekers who can claim a near-term handoff instead of circling blindly.
- **Direct handoff coordination** between two matched drivers with shared timing and vehicle identification.
- **Potential reduction of unnecessary driving** when a match succeeds (not measured in-app).
- **Incentives** via credits that move only after verified completion.
- **Network effects (potential):** more publishers make seeking more useful and vice versa — **not measured** in the MVP.

Switch It does **not** claim to eliminate parking scarcity or to legally protect a public space.

Business goals

MVP goals (current)

- Deliver a working end-to-end handoff flow on the web (auth → vehicle → publish/claim → timing → verify → credits → history).
- Keep critical race conditions and credit consistency in PostgreSQL.
- Deploy as a usable phone-first PWA-style web app.
- Be honest about public-parking limitations.

Possible future business goals (NOT CURRENTLY IMPLEMENTED)

- Paid subscriptions, city partnerships, or advertising.
- Ratings, no-show penalties, or stronger anti-fraud / unique-identity controls.
- In-app turn-by-turn navigation / ETA.
- City-scale geo search (e.g. PostGIS).

Core software capabilities

Verified in the repository:

Capability	Status
Email/password authentication (Supabase Auth)	Implemented
Email confirmation before first login (Confirm email)	Implemented
Forgot password / reset via email (Login only; no Profile change-password)	Implemented
Shared password policy (8+, upper/lower/digit/special; max 72)	Implemented (signup + reset; Auth enforces)
Profile (display name, credits display)	Implemented
Header credit balance (same <code>profiles.credits</code> as Profile)	Implemented
Vehicle setup / onboarding	Implemented
Publish parking spot (location + departure 0–10 min)	Implemented
Map discovery (available spots)	Implemented
Claim with location + distance check	Implemented

Handoff timing (Now, future, auto-start, I'm leaving now, 3+2 window)	Implemented
External navigation chooser	Implemented
Live seeker location during active handoff	Implemented (mandatory in active-claim UI; permission/GPS dependent). Web and native both post through Edge Function <code>handoff-seeker-location</code> . Web GPS is foreground-only; native can continue in the background.
Completion success overlay + return to Find parking	Implemented (overlay lives at app root; <code>/map</code> is prepared underneath; MapLibre is not remounted if already on <code>/map</code>)
Cancel / release / expiry terminal overlay	Implemented (role-specific copy; does not change credit rules)
Cancellation / release with structured reasons	Implemented
Publisher plate-suffix verification + attempt logout	Implemented
Credits (check on claim; transfer on complete)	Implemented
History with pagination	Implemented
Help & Safety content	Implemented
PWA manifest / offline fallback page	Implemented
Stale-client recovery after a new deploy	Implemented (one hard reload for known chunk/RSC failures; unknown errors still show the error page)
Push notifications	Optional / experimental: infrastructure exists in the repo, but end-to-end production delivery (especially native iOS/APNs) has not been fully configured and verified. Not required for the core web MVP.
Payments / chat / ratings	Not currently implemented
User-uploaded vehicle photos	Not currently implemented (removed; CarImages catalog used)
Spoken verification codes as product UX	Not currently implemented (legacy/dormant DB remnants; plate suffix is current)

Main user flows

A. Publisher — happy path

1. Register (verify email) / login (Forgot password available).
2. Complete vehicle onboarding if incomplete.

3. **Share a spot** → pick/confirm location → choose departure → publish.
4. Wait for seeker claim (Realtime / refresh updates UI).
5. At `available_at` , if claimed: active 3-minute window auto-starts. If unclaimed: listing expires at departure.
6. Optionally press **I'm leaving now** earlier:
 - o If claimed: start that handoff immediately.
 - o If unclaimed: convert to a live **Now-style** 3-minute claimable window (listing stays `available`).
7. Optionally **Wait 2 more min** once after a seeker is claimed and the live window has started (hard cap: start + 5 minutes).
8. Enter seeker's last 2 plate digits → **Confirm handoff**.
9. Credits: seeker -1, publisher +1; both see History entry.

B. Seeker — happy path

1. Register (verify email) / login + vehicle.
2. **Find parking** → select pin → **I'm on my way**.
3. Fresh location required at claim; must be within 1500 m.
4. Navigate; live location sharing starts for the active claim (if permission/GPS allow). GPS is posted through Edge Function `handoff-seeker-location` .
5. Arrive; publisher confirms; success overlay; Find parking is prepared underneath; credits debit on success.

C. Important alternative flows

Flow	Behavior (current)
Seeker releases before live start (future listing)	Claim cancelled; spot may reopen as <code>available</code> for other seekers; original timing remains; releasing seeker cannot reclaim the same <code>parking_spot_id</code>
Seeker releases after handoff started	Exchange ends; spot does not automatically relist; no credits
Same seeker tries to reclaim a spot they voluntarily released	Rejected (<code>ALREADY_RELEASED_THIS_SPOT</code>)
Two seekers race to claim	One wins via DB lock + unique active-claim-per-spot
Insufficient credits	Claim rejected; no debit attempted
Wrong plate digits	Attempts counted; after 3 failures, temporary lock (~2 min)
Publisher cancels	Spot/claim terminal with structured reason; no credits
Expiry	Spot/claim expire; no credits
Double complete / retry	Completion designed to be idempotent when already completed with consistent ledger

Timing model (canonical)

Mode	Behavior
Now	handoff_started_at set immediately; expires_at ≈ start + 3 minutes; listing may stay available until claimed; a later claim gets remaining time
Future	available_at = promised departure; handoff_started_at null until start
Future + claimed	At available_at, auto-start 3-minute live window
Future + unclaimed	Listing expires at available_at; cannot claim afterward
I'm leaving now + claimed	Live 3-minute window from actual early start
I'm leaving now + unclaimed	Stays available; sets handoff_started_at / expires_at = now + 3; later claim does not reset timer
Extension	Once, +2 minutes, publisher-controlled, only when claimed/live; hard cap start + 5 minutes

Credits (canonical)

No credits move on publish, claim, start, extension, cancellation, or expiry.

Transfer only on successful `complete_claim` : seeker -1, publisher +1.

Product limitations

1. **Public parking cannot be reserved.** Another street driver may take the spot.
 2. **No GPS proximity requirement to complete** the handoff — publisher confirmation is the gate.
 3. **GPS accuracy / permission** issues can block claiming or live location.
 4. **Network / Realtime** issues can delay UI updates (DB remains source of truth).
 5. **Fraud / collusion / multi-account credit farming** — only partially mitigated; the app does not prove unique real-world identity.
 6. **MapTiler / CarImages** dependency for maps and vehicle imagery.
 7. **No payments, chat, ratings, or legal reservation.**
 8. Discovery is **not** a sophisticated geo-index query; claim distance is enforced at claim time.
 9. **Push** is optional/experimental and not part of the verified core web MVP.
-

Repository references

- `README.md`
- `src/app/map/page.tsx` , `src/app/spots/new/page.tsx`
- `src/actions/spots.ts` , `src/actions/claims.ts` , `src/actions/auth.ts`
- `src/lib/spots/constants.ts` , `src/lib/handoff/cancellation-reasons.ts`
- `src/components/help/HelpSafetyContent.tsx`
- `src/components/layout/HeaderCreditsBalance.tsx`
- `src/components/handoff/HandoffCompletionSuccessController.tsx`
- `src/components/handoff/HandoffTerminalEndedController.tsx`
- `src/lib/navigation/stale-client-build.ts`

- `src/lib/location/publish-seeker-live-location.ts`
- `supabase/functions/handoff-seeker-location/index.ts`
- `supabase/migrations/20260819140000_unclaimed_early_start_live_window.sql`
- `supabase/migrations/20260819130000_prevent_seeker_reclaim.sql`
- `supabase/migrations/20260818170000_publisher_verifies_seeker_plate.sql`
- `supabase/migrations/20260818200000_drop_vehicle_photos.sql`